Chat history

You said:

Now, we are doing Special Participation B: ChatGPT on Homework 12 coding.

ChatGPT said:

Great — we're in **Special Participation B: ChatGPT on Homework 12 coding** mode now.

Go ahead and paste **the relevant parts of Homework 12**, or tell me:

- Which question you're currently on
- The file(s) you need to edit
- What error messages (if any) you're seeing
- And whether this is in the **NumPy**, **PyTorch**, or **mixed** portion of the assignment

I'll switch into the precise EECS 182 coding-assistant style and help you implement, debug, or reason through whatever you need.

You said:

hw12.pdf PDF
Here is the PDF. There is one coding question on this homework, and it is a debugging one. Do not answer it yet.

ChatGPT said:

Perfect — I've read **Homework 12**, and I see the **one coding question** is the debugging question in **Problem 1: Debugging Transformers**. I won't answer it yet, exactly as you requested.

Whenever you're ready, tell me:

- **What part you want to start with**,
- Or paste the **buggy snippet you're investigating**,
- Or ask for **help walking through the logic before giving an answer**.

I'll stay in full EECS 182 debugging mode.

You said:

OK, work through Question 1 in excruciating detail. First, briefly restate the question. Second, provide some helpful hints that guide a problem solver through the question. Finally, provide a fully written solution of your own with detailed but intuitive and accessible reasoning.

ChatGPT said:

Here's Q1 unpacked carefully.

1. Brief restatement of the question

You have a Transformer seq2seq model with:

- Token embeddings `emb_word` and positional embeddings `emb_pos`
- A tied output head: `lm_head.weight = self.emb_word.weight`
- Manual initialization:

```
self.emb_word.weight.data.normal_(mean=0, std=1)
self.emb_pos.weight.data.normal_(mean=0, std=1)
```

After training, your model's loss (~23.4) is much worse than your friend's (~6.1). You're told the bug is in **how the head is initialized**, and that the fix must affect **line 9**, where `emb_word` is initialized (since that indirectly sets the head's weights via tying). You need to:

1. Identify the bug in the initialization.
2. Show the corrected lines of code.
3. Explain why the fix makes sense, especially given that `d_model` is large.

---

2. Hint-style guidance (without immediately giving the full answer)

Use these as a roadmap if you want to solve it yourself.

Hint 1: How are logits computed?

- The output logits for each vocab word come from a dot product:

  $\text{logit}_i = h^\top w_i$

  where:

  - $h \in \mathbb{R}^{d_{\text{model}}}$ is the final hidden state,
  - $w_i \in \mathbb{R}^{d_{\text{model}}}$ is the i-th row of `lm_head.weight` (tied to the embedding).

Think about what happens to the **magnitude** of this dot product when both vectors are randomly initialized with entries of variance 1 and **d_model is large**.

Hint 2: Variance of a dot product

Very roughly:

- If h_j and w_{ij} are independent, zero-mean with variance σ_h^2 and σ_w^2,
- Then the variance of $h^\top w_i = \sum_{j=1}^{d_{\text{model}}} h_j w_{ij}$ is on the order of:

  $\text{Var}(h^\top w_i) \approx d_{\text{model}} \sigma_h^2 \sigma_w^2$

So if both variances are ~1, the variance of each logit becomes ~d_{model}. For typical Transformers d_{model} is in the hundreds or thousands.

Ask yourself:

- What does a **softmax** do when logits have magnitude ~20, 30, 40…?
- What does that do to the gradients backpropagated through the softmax and cross entropy?

Hint 3: Why does tying the head to `emb_word` matter?

- You explicitly set:
- `self.lm_head.weight = self.emb_word.weight`

- That means the initialization of the **head** is *exactly* the initialization of `emb_word`.
- You don't directly re-initialize `lm_head.weight` later — its init is effectively determined at **line 9**.
- So if you want to fix the head's initialization, you need to fix how `emb_word.weight` is initialized.

Hint 4: What should the scale be?

Usually for linear layers or embeddings feeding into a dot product, we want the variance of the output to be roughly constant, not to **grow with d_model**.

Standard trick:

- Make the per-component variance scale like 1/dmodel1/dmodel, so the dot product's variance is O(1)O(1).
- That means a **standard deviation of roughly 1/dmodel1/ dmodel**
- , not 1.

Think:

- What if you change:

`std=1`

to something like:

- `std=1 / math.sqrt(d_model)`

Now the logits won't blow up with larger `d_model`, the softmax will be better behaved, and the gradients won't vanish or explode so badly.

---

3. Full solution with detailed but accessible reasoning

The bug (conceptual explanation)

The bug is that the **embedding (and therefore the output head) is initialized with too large a variance independent of `d_model`**:

`self.emb_word.weight.data.normal_(mean=0, std=1)`

Because `lm_head.weight` is tied to `emb_word.weight`, this also initializes the **output layer weights** with `N(0, 1)` entries.

For each vocab word ii, the logit is:

logiti=h⊤wi=∑j=1dmodelhjwijlogiti = h⊤wi = j=1∑dmodel hjwij

If we assume:

- hjhj has variance on the order of 1 (very rough but reasonable at init / early training),
- wij∼N(0,1)wij ∼ N(0, 1),

then:

Var(logiti)≈dmodel⋅1⋅1=dmodelVar(logiti) ≈ dmodel ⋅ 1 ⋅ 1 = dmodel

So the standard deviation of each logit scales like:

dmodel dmodel

For a typical `d_model` like 512 or 1024:

- 512≈22.6 512
- ≈ 32

That means logits commonly land in the range of ±20 or ±30. The softmax of such logits will be extremely peaked: the largest logit dominates so much that:

- One token gets probability ≈ 1.
- All other tokens get probability ≈ 0.
- Cross-entropy gradients almost vanish for most classes, and can become numerically unstable.

This makes training very hard and can lead to terrible loss (which matches the plot where your model's loss stays up at ~23, way worse than your friend's).

So the bug is: **initializing the tied embedding / head with std=1 causes logits to have variance proportional to `d_model`, which is way too large for large `d_model`, saturating the softmax and killing learning.**

The fix (code)

We want the variance of the dot product to be roughly constant as `d_model` grows.

To achieve roughly unit variance for $h^\top w_i h^\top w_i$, we want:

$\mathrm{Var}(w_{ij}) \approx \frac{1}{d_{model}} \mathrm{Var}(w_{ij}) \approx d_{model}1$

so that:

$\mathrm{Var}(h^\top w_i) \approx d_{model} \cdot 1 \cdot \frac{1}{d_{model}} = 1 \mathrm{Var}(h^\top w_i) \approx d_{model} \cdot 1 \cdot d_{model}1 = 1$

This means we should initialize `emb_word.weight` with **standard deviation $1/\sqrt{d_{model}}$ $1/\sqrt{}$ d_{model}**

instead of 1.

A minimal fix (importing `math` somewhere) could be:

```
import math

class Transformer(nn.Module):
    def __init__(self, n_words, max_len, n_layers,
                 d_model, n_heads, d_ffn, p_drop):
        super().__init__()
        self.emb_word = nn.Embedding(n_words, d_model)
        self.emb_pos = nn.Embedding(max_len, d_model)

        # Initialize embedding layers with scale that accounts for d_model
        self.emb_word.weight.data.normal_(mean=0.0, std=1.0 / math.sqrt(d_model))
        self.emb_pos.weight.data.normal_(mean=0.0, std=1.0 / math.sqrt(d_model))

        self.emb_ln = nn.LayerNorm(d_model)
        ...
        self.lm_head = nn.Linear(d_model, n_words)
        # Share lm_head weights with emb_word
        self.lm_head.weight = self.emb_word.weight
        self.criterion = nn.CrossEntropyLoss(ignore_index=-100)
```

The key corrected line (the one the problem cares about) is:

```
self.emb_word.weight.data.normal_(mean=0.0, std=1.0 / math.sqrt(d_model))
```

Optionally doing the same for `emb_pos` is nice for consistency.

Why this fix works (intuitive reasoning)

1. **Head weights are tied to `emb_word`.**
   `self.lm_head.weight = self.emb_word.weight` makes them the *same tensor*. Whatever initialization you use for `emb_word` is used for the output head.
2. **We control logit scale through embedding scale.**

For each output token, the logit is a dot product of:

- The hidden vector hh,
- A row of the weight matrix wiwi (which is just the embedding of token i).

If each component of wiwi has std=1, the variance of the logit grows linearly with `d_model`. Large `d_model` → huge logits → softmax saturates → vanishing/exploding gradients.

3. **Scaling by 1/dmodel1/ dmodel**

keeps logits well-behaved.
By choosing `std = 1 / sqrt(d_model)`:

- Each component of wiwi is small.
- The dot product's variance is roughly:

  dmodel·Var(hj)·1dmodel≈Var(hj)dmodel · Var(hj) · dmodel1 ≈ Var(hj)

- So the logits have magnitude O(1)O(1) instead of O(dmodel)O(dmodel

3. ○).

4. **Softmax works in a "comfortable" regime.**
   With logits around ±1 or ±2:
   - Softmax doesn't saturate as badly.
   - Probabilities are more spread out.
   - Gradients are healthy and propagate through the head and back into the model.
5. **Result: better optimization, lower loss.**
   Your friend's curve (loss ~6) likely corresponds to such a well-scaled initialization. Once you fix the scaling on the tied embedding/head, your model's training dynamics should look much more like theirs.

Final succinct answers in the format the HW asks for

The bug (brief description):
The word embedding (and thus the tied output head) is initialized with `std=1` independent of `d_model`. Since the output logits are dot products between hidden states and these weights, their variance grows with `d_model`, producing extremely large logits that saturate the softmax and make training fail.

The fix (code):
(Show only the changed lines.)

```
import math

# Initialize embedding layers with variance that scales with d_model
self.emb_word.weight.data.normal_(mean=0.0, std=1.0 / math.sqrt(d_model))
self.emb_pos.weight.data.normal_(mean=0.0, std=1.0 / math.sqrt(d_model))
```

(Everything else stays the same, including the weight tying.)

Why the fix should work (brief explanation):
With `std=1`, the logits from the tied head have variance ≈ `d_model`, so for large `d_model` they are huge, causing the softmax to be almost one-hot and gradients to vanish or explode. Scaling the embedding/head weights by `1 / sqrt(d_model)` keeps the logit variance roughly constant as `d_model` grows, so the softmax operates in a non-saturated regime and the model can actually learn, leading to much better loss like your friend's.

You said:

H12Q4_Copy_of_q_vae.ipynb File
utils.py Python
vae.py Python
My bad! There is also an additional coding question; Question 4. I uploaded the relevant files: the two python files containing the functions we are told to modify, and the .ipynb notebook. Now, walk me through implementation.

ChatGPT said:

Nice, this is a fun one. Let's go step by step and wire the math to the code.

I'll break it into:

1. What Question 4 is asking conceptually
2. Implementing `sample_gaussian` in `utils.py`
3. Implementing `negative_elbo_bound` in `vae.py`
4. Sanity checks / how to think about the outputs

---

1. What Q4 wants you to do

From the PDF, Q4 is the VAE problem: they define the ELBO, explain the reparameterization trick, and then explicitly say:

> "You will implement the reparameterization trick in `q_vae_gan.ipynb` at the function `sample_gaussian`. Specifically, your answer will take in the mean `m` and variance `v` of the Gaussian and return a sample `x ~ N(m, diag(v))`. Then, implement `negative_elbo_bound` loss function."

Your codebase instead has you implement:

- `sample_gaussian(m, v)` in **`utils.py`**

`negative_elbo_bound(self, x)` in **`vae.py`**

And then you train with `python experiment.py --model vae`.

High-level math:

- Prior: $p(z)=N(0,I)p(z) = N(0, I)$
- Encoder: $qϕ(z|x)=N(μϕ(x),diag(σϕ2(x)))qϕ(z|x) = N(μϕ(x), diag(σϕ2(x)))$
- Decoder: $pθ(x|z)=Bernoulli(x|fθ(z))pθ(x|z) = Bernoulli(x|fθ(z))$, implemented via logits.

The **ELBO** for one sample x is:

$ELBO(x)=Eq(z|x)[logp(x|z)]−DKL(q(z|x)‖p(z))ELBO(x) = Eq(z|x)[\log p(x|z)] − DKL(q(z|x)‖p(z))$

We **minimize** the **negative** ELBO:

$negative\_ELBO(x)=−ELBO(x)=DKL(q(z|x)‖p(z))−Eq(z|x)[logp(x|z)]negative\_ELBO(x) = −ELBO(x) = DKL(q(z|x)‖p(z)) − Eq(z|x)[\log p(x|z)]$

And we estimate the expectation with **one Monte Carlo sample** of z.

---

2. Implementing `sample_gaussian` (reparameterization trick)

Open **`utils.py`**, find:

```
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # TODO: Modify/complete the code here
```

```
    # Sample z
    ################################################################################

    ################################################################################
    # End of code modification
    ################################################################################
    eps = torch.randn_like(v)
    z = m + eps * torch.sqrt(v)
    return z
```

Conceptually, we want to sample:

$z \sim N(m, \text{diag}(v))$

Direct sampling `z = torch.normal(m, torch.sqrt(v))` would break backprop because the randomness is inside the distribution parameters. The **reparameterization trick** instead writes:

$z = m + v \cdot \epsilon, \quad \epsilon \sim N(0, I)$

Where `eps` is standard normal noise **independent of m and v** (so gradients can flow into `m` and `v` through the deterministic transformation).

So the implementation is literally:

```
eps = torch.randn_like(v)   # ε ~ N(0, I) with same shape as v
z = m + eps * torch.sqrt(v) # elementwise m + sqrt(v)*ε
return z
```

Inside the TODO block, your final code should be:

```
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    ################################################################################
    # TODO: Modify/complete the code here
    # Sample z
    ################################################################################
    eps = torch.randn_like(v)         # Sample epsilon ~ N(0, I)
    z = m + eps * torch.sqrt(v)       # Reparameterization: z = m + sqrt(v) * eps
    ################################################################################
    # End of code modification
    ################################################################################
    return z
```

(If your file already has those two lines *after* the TODO comment, you can just move them into the TODO region or leave as-is; the logic is exactly what the assignment wants.)

Shape-wise:

- `m` and `v` are `(batch, z_dim)` (or possibly `(batch, ..., z_dim)`).
- `torch.randn_like(v)` has the same shape.
- The operations are elementwise, so `z` has the same shape as `m` and `v`.

---

3. Implementing `negative_elbo_bound` in `vae.py`

Now open **`vae.py`** and look at `VAE.negative_elbo_bound`:

```
class VAE(nn.Module):
```

```
    ...

    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)


        # compute metrics
        kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
        rec = -ut.log_bernoulli_with_logits(x, logits)
        nelbo = kl + rec

        nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()

        ###############################################################################
        # TODO: Modify/complete the code here
        # Compute negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs should all be scalar
        ###############################################################################

        ###############################################################################
        # End of code modification
        ###############################################################################
        return nelbo, kl, rec
```

Let's read what's already there:

- `self.enc.encode(x)` returns `(mean, var)` for the approximate posterior $q(z|x)$q(z|x).
- `ut.sample_gaussian(mean, var)` samples zz from that Gaussian.
- `self.dec.decode(z)` returns logits for Bernoulli pixels (i.e. $\log p(x=1|z)\log p(x=1|z)$ $1-p(x=1|z)1-p(x=1|z)$).
- `ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])` gives the **KL divergence** $D_{KL}(q(z|x) \| p(z))$DKL(q(z|x)∥p(z)) per sample (shape: `(batch,)`).
- `ut.log_bernoulli_with_logits(x, logits)` gives $\log p(x|z)$logp(x|z) per sample, so
- `rec = -ut.log_bernoulli_with_logits(x, logits)`

  is **-log-likelihood** (the reconstruction *loss* term) per sample.
- `nelbo = kl + rec` is exactly:

  negative ELBO$_n$=KL$_n$+reconstruction_loss$_n$negative ELBOn = KLn + reconstruction_lossn

  for each sample n in the batch.

So the only missing step is: **average over the batch** and return **scalars**.

They actually already wrote:

```
nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()
```

This turns `(batch,)` vectors into scalar tensors (shape `()`), exactly what the docstring asks for.

So the full, correct implementation is:

```
def negative_elbo_bound(self, x):
    """
    Computes the Evidence Lower Bound, KL and, Reconstruction costs

    Args:
        x: tensor: (batch, dim): Observations

    Returns:
        nelbo: tensor: (): Negative evidence lower bound
        kl: tensor: (): ELBO KL divergence to prior
        rec: tensor: (): ELBO Reconstruction term
    """
    # Encode to get q(z|x) parameters
    mean, var = self.enc.encode(x)

    # Sample z via reparameterization trick
    z = ut.sample_gaussian(mean, var)

    # Decode to get Bernoulli logits for p(x|z)
    logits = self.dec.decode(z)

    # KL term: KL(q(z|x) || p(z))
    kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])
    # Reconstruction term: -log p(x|z)
    rec = -ut.log_bernoulli_with_logits(x, logits)

    # Negative ELBO per sample
    nelbo = kl + rec

    # Average over batch to get scalars
    nelbo = nelbo.mean()
    kl = kl.mean()
    rec = rec.mean()

    return nelbo, kl, rec
```

That already matches the code you pasted (the stuff above the TODO). So in your actual homework skeleton, you probably just need to:

- Add the `kl`, `rec`, `nelbo` computations,
- Average them,
- And return them.

Conceptually:

- `kl` → how much your latent distribution deviates from the prior (regularization).
- `rec` → how badly you reconstruct the input (data fit).
- `nelbo = kl + rec` → a single scalar "loss" you minimize.

Then `loss(self, x)` just picks `nelbo`:

```
def loss(self, x):
    nelbo, kl, rec = self.negative_elbo_bound(x)
    loss = nelbo

    summaries = dict((
        ('train/loss', nelbo),
        ('gen/elbo', -nelbo),
        ('gen/kl_z', kl),
        ('gen/rec', rec),
    ))
```

```
    return loss, summaries
```

---

4. Sanity checks / intuition

After implementing both:

- Run `python experiment.py --model vae` or the notebook cell that trains the VAE.
- Over training:
    - `train/loss` (NELBO) should go **down**.
    - `gen/elbo = -nelbo` should go **up**.
    - `gen/kl_z` and `gen/rec` will typically trade off (KL up, rec down, or vice versa).

Intuition:

- If KL is **too small** and rec is **too large**, the model might be ignoring the prior (posterior collapse in the opposite direction).
- If KL is **large** and rec is **small**, latent codes are too far from the prior and generation from the prior will be bad.
- A "reasonable" solution balances both.

If you want, next step we can go line-by-line through what happens in one forward pass with a single minibatch and interpret each tensor's shape & meaning.

You said:

hw12.pdf PDF
So far, so good. We have one more coding question to work through together: Question 5. Before we get to the Jupyter notebook, work me through 5(c) and provide code for it. We will do the Jupyter notebook next; I'll upload it. To refresh your memory, I re-uploaded the Homework 12 pdf.

ChatGPT said:

Nice, this is a fun one. Let's zoom in on **5(c)** and build both the math intuition and the code you'd drop into a notebook or script.

I'll do:

1. Restate 5(c) in my own words
2. Work through the objective and gradient behavior
3. Answer "what does β go to?"
4. Give you a clean, ready-to-run Python snippet that shows the convergence behavior

All of this is just based on the Homework 12 statement.

---

1. What 5(c) is asking

From the PDF, the simplified toy setup is:

- Two features: a "true" feature $\phi_t(x)$ and an "alias" feature $\phi_a(x)$.
- At **training** time: $\phi_t(x) = \phi_a(x)$ at the single training point, so they're indistinguishable.
- At **test** time: ϕ_t and ϕ_a are orthonormal wrt the test distribution.
- Feature weights: $\mathbf{c = (c_0, c_1)}$ (slow weights).
- Learner does minimal-norm regression and you've already shown in 5(a) that:

  β^=1c02+c12[c0c1]β^ = c02 + c121 [c0c1]

In 5(c), you are asked to:

> "Generate a plot showing that, for some initialization c(0)c(0), as the number of iterations i→∞i → ∞ the weights empirically converge to c0=‖c(0)‖c0 = ‖c(0)‖, c1=0c1 = 0 using gradient descent with a sufficiently small step size. Include the initialization and its norm and the final weights. What will β go to?"

So you need:

- The **test loss** as a function of c_0, c_1
- Its **gradient** (from 5(b))
- A gradient descent loop on c
- A plot or at least numbers showing convergence to (‖c(0)‖,0)(‖c(0)‖, 0)
- And the limit of β^ at that point

---

2. The test loss $J(c_0, c_1)$

Target test function is:

fα(x)=ϕt(x)fα(x) = ϕt(x)

Your learned function (using min-norm regression and feature weights c) is:

f^β,c(x)=β^0c0ϕt(x)+β^1c1ϕa(x)f^β,c(x) = β^0c0ϕt(x) + β^1c1ϕa(x)

Given

β^0=c0c02+c12,β^1=c1c02+c12β^0 = c02 + c12c0, β^1 = c02 + c12c1

Define:

S=c02+c12S = c02 + c12

Then:

- β^0c0=c02Sβ^0c0 = Sc02
- β^1c1=c12Sβ^1c1 = Sc12

So:

f^β,c(x)=c02Sϕt(x)+c12Sϕa(x)f^β,c(x) = Sc02ϕt(x) + Sc12ϕa(x)

The **test-time error** at x is:

err(x)=fα(x)−f^β,c(x)=ϕt(x)−c02Sϕt(x)−c12Sϕa(x)err(x) = fα(x) − f^β,c(x) = ϕt(x) − Sc02ϕt(x) − Sc12ϕa(x)

Let

α=c02S,β=c12Sα = Sc02, β = Sc12

Then:

err(x)=(1−α)ϕt(x)−βϕa(x)err(x) = (1 − α)ϕt(x) − βϕa(x)

Squared error:

err(x)2=(1−α)2ϕt(x)2+β2ϕa(x)2−2(1−α)βϕt(x)ϕa(x)err(x)2 = (1 − α)2ϕt(x)2 + β2ϕa(x)2 − 2(1 − α)βϕt(x)ϕa(x)

Under the **test distribution** μ_{test}, we assume orthonormality:

- E[ϕt2]=1E[ϕt2] = 1

- $E[\phi a2]=1E[\phi a2] = 1$
- $E[\phi t\phi a]=0E[\phi t\phi a] = 0$

So the **expected squared error** is:

$E[err2]=(1-\alpha)2+\beta 2E[err2] = (1 - \alpha)2 + \beta 2$

The expected **test loss** (with the ½ factor) is:

$J(c0,c1)=12[(1-\alpha)2+\beta 2]J(c0, c1) = 21[(1 - \alpha)2 + \beta 2]$

Plugging $\alpha=c02S\alpha = Sc02$, $\beta=c12S\beta = Sc12$, and simplifying, you get a very clean expression:

$J(c0,c1)=c14(c02+c12)2J(c0, c1) = (c02 + c12)2c14$

So:

- The loss depends **only on c_1 and the radius**, not on the sign of c_0.
- Intuitively: only the **alias** feature weight matters for error; true feature weight just scales in a benign way.

---

3. Gradient of $J(c_0, c_1)$ and its behavior

From 5(b) (or by explicit differentiation), the gradients are:

$\partial J\partial c0=-4c0c14(c02+c12)3\partial c0\partial J = -(c02 + c12)34c0c14$ $\partial J\partial c1=4c02c13(c02+c12)3\partial c1\partial J = (c02 + c12)34c02c13$

Some very important structure pops out:

3.1. Norm of c is preserved under gradient flow

Let $R2=c02+c12R2 = c02 + c12$. Consider continuous-time gradient flow:

$c\dot{}0=-\partial J\partial c0, c\dot{}1=-\partial J\partial c1c\dot{}0 = -\partial c0\partial J, \quad c\dot{}1 = -\partial c1\partial J$

Then:

$ddt(c02+c12)=2c0c\dot{}0+2c1c\dot{}1=-2(c0\partial J\partial c0+c1\partial J\partial c1)dtd(c02 + c12) = 2c0c\dot{}0 + 2c1c\dot{}1 = -2(c0\partial c0\partial J + c1\partial c1\partial J)$

If you plug in the expressions for $\partial J/\partial c_0$ and $\partial J/\partial c_1$, all terms cancel out and you get:

$ddt(c02+c12)=0dtd(c02 + c12) = 0$

So the **radius $\|c\|\|c\|$** is **constant** during gradient descent (with sufficiently small step size, GD approximates this).

That means the trajectory of c lies on the **circle** of radius $\|c(0)\|\|c(0)\|$.

3.2. Direction of c rotates toward the true feature

The loss is:

$J=c14(c02+c12)2J = (c02 + c12)2c14$

- If $c1=0c1 = 0$, then $J=0J = 0$ (perfect prediction).
- If $c0=0c0 = 0$, then $J=1J = 1$ (worst case, all weight on alias).
- For any $c0\neq 0c0 \neq 0$, $c1\neq 0c1 \neq 0$, gradients are non-zero: the system will move.

Gradient descent on J:

c0(i+1)=c0(i)−η∂J∂c0(c(i))c0(i+1) = c0(i) − η∂c0∂J(c(i)) c1(i+1)=c1(i)−η∂J∂c1(c(i))c1(i+1) = c1(i) − η∂c1∂J(c(i))

- Because the radius is preserved, c(i)c(i) stays on the circle ‖c‖=‖c(0)‖‖c‖ = ‖c(0)‖.
- The gradient direction is such that **c_1 is driven toward 0** and c_0 is pushed toward $\pm\|c^{(0)}\|$.

So with a small enough step size η, as i→∞i → ∞:

c0(i)→‖c(0)‖,c1(i)→0c0(i) → ‖c(0)‖, c1(i) → 0

(If you start with $c_0 > 0$; if $c_0 < 0$ you converge to $-\|c(0)\|$−‖c(0)‖, but the problem statement assumes the positive case.)

---

4. What does β go to?

Recall:

β^=1c02+c12[c0c1]β^ = c02 + c121 [c0c1]

At convergence, we have:

- c0*=‖c(0)‖=Rc0* = ‖c(0)‖ = R,
- c1*=0c1* = 0,
- so S*=(c0*)2+(c1*)2=R2S* = (c0*)2 + (c1*)2 = R2.

Then:

β^0*=c0*S*=RR2=1Rβ^0* = S*c0* = R2R = R1 β^1*=0R2=0β^1* = R20 = 0

So:

β^*=[1/‖c(0)‖0]β^* = [1/‖c(0)‖0]

The actual learned function uses β^0c0ϕtβ^0c0ϕt, and at the limit:

β^0*c0*=1R·R=1β^0*c0* = R1 · R = 1

So the learner represents exactly fα(x)=ϕt(x)fα(x) = ϕt(x) with **zero error**, as desired.

---

5. Code for 5(c): gradient descent on c and plotting convergence

Here's a minimal, clean Python script (NumPy + Matplotlib) that:

- Chooses an initialization c(0)c(0),
- Runs gradient descent using the analytic gradients,
- Prints initial c, its norm, and final c,
- Plots c_0 and c_1 over iterations.

```
import numpy as np
import matplotlib.pyplot as plt

# ----- Objective and gradients from the derivation -----
def loss_J(c0, c1):
    """
    J(c0, c1) = c1^4 / (c0^2 + c1^2)^2
    """
    S = c0**2 + c1**2
    return (c1**4) / (S**2)
```

```
def grad_J(c0, c1):
    """
    Gradients:
        dJ/dc0 = -4 * c0 * c1^4 / (c0^2 + c1^2)^3
        dJ/dc1 =  4 * c0^2 * c1^3 / (c0^2 + c1^2)^3
    """
    S = c0**2 + c1**2
    dJ_dc0 = -4.0 * c0 * (c1**4) / (S**3)
    dJ_dc1 =  4.0 * (c0**2) * (c1**3) / (S**3)
    return dJ_dc0, dJ_dc1

# ----- Gradient descent on c -----
# Choose some non-degenerate initialization
c0 = 0.6
c1 = 0.8
c0_init, c1_init = c0, c1

R = np.sqrt(c0_init**2 + c1_init**2)  # initial norm

eta = 0.05            # sufficiently small step size
num_iters = 2000

history_c0 = []
history_c1 = []
history_loss = []

for i in range(num_iters):
    history_c0.append(c0)
    history_c1.append(c1)
    history_loss.append(loss_J(c0, c1))

    dJ_dc0, dJ_dc1 = grad_J(c0, c1)
    # Gradient descent update
    c0 = c0 - eta * dJ_dc0
    c1 = c1 - eta * dJ_dc1

# Final values
c0_final, c1_final = c0, c1
R_final = np.sqrt(c0_final**2 + c1_final**2)

print("Initial c:", (c0_init, c1_init))
print("Initial norm ||c(0)||:", R)
print("Final c after gradient descent:", (c0_final, c1_final))
print("Final norm ||c_final|| (should be ~ same as initial):", R_final)
print("Target limiting point: (||c(0)||, 0) = ({:.4f}, 0.0)".format(R))
print("Final loss J(c_final):", loss_J(c0_final, c1_final))

# ----- Plots -----
fig, axs = plt.subplots(1, 2, figsize=(12, 4))

# Plot c0, c1 over iterations
axs[0].plot(history_c0, label="c0")
axs[0].plot(history_c1, label="c1")
axs[0].axhline(R, color="gray", linestyle="--", label="||c(0)||")
axs[0].set_xlabel("Iteration")
axs[0].set_ylabel("c value")
axs[0].set_title("Evolution of c0 and c1")
axs[0].legend()

# Plot trajectory in (c0, c1) plane
axs[1].plot(history_c0, history_c1, marker="", linestyle="-")
circle = plt.Circle((0, 0), R, fill=False, linestyle="--")
axs[1].add_artist(circle)
axs[1].scatter([c0_init], [c1_init], color="red", label="init")
axs[1].scatter([c0_final], [c1_final], color="green", label="final")
axs[1].set_xlabel("c0")
axs[1].set_ylabel("c1")
```

```
axs[1].set_aspect("equal", "box")
axs[1].set_title("Trajectory of c in parameter space")
axs[1].legend()

plt.tight_layout()
plt.show()
```

What this code demonstrates

- **Norm preservation**:
  `R_final` will be numerically very close to `R`, showing that the dynamics stay on the circle of radius $\|c(0)\|$.
- **Convergence**:
  `c1_final` goes very close to 0, and `c0_final` goes very close to `R`.
- **Loss**:
  `loss_J(c0_final, c1_final)` → 0, reflecting perfect prediction.

And from this you can explicitly read off the answers the problem wants:

- The empirical convergence $c_0 \to \|c^{(0)}\|$, $c_1 \to 0$,
- And the limiting $\hat{\beta}$:

β^*=[1/‖c(0)‖0]β^* = [1/‖c(0)‖0]

If you want, next step we can adapt this code into whatever skeleton they give you in the Jupyter notebook for the meta-learning demo.

You said:

Here were the results of the code: Initial c: (0.6, 0.8) Initial norm ||c(0)||: 1.0 Final c after gradient descent: (1.010025329822829, 0.036203934259138384) Final norm ||c_final|| (should be ~ same as initial): 1.0106739789563963 Target limiting point: (||c(0)||, 0) = (1.0000, 0.0) Final loss J(c_final): 1.6465644434097214e-06

ChatGPT said:

Nice, that looks exactly like what we expect (up to discrete-time GD quirks).

Let's interpret your numbers:

- **Initial c:** (0.6, 0.8)
- **‖c(0)‖:** 1.0
- **Final c:** (~1.01, ~0.036)
- **‖c_final‖:** ~1.0107
- **Final loss J(c_final):** ~1.6×10^{-6} (basically 0)

What this shows:

1. **c_1 is being driven toward 0.**
   It starts at 0.8 and ends near 0.036, and if you ran longer or used a smaller step size it would get arbitrarily close to 0.
2. **c stays on (or very close to) a circle.**
   In exact gradient flow the radius is constant; with discrete steps and η = 0.05 you get a tiny drift, hence 1.01 instead of 1. If you make `eta = 0.01`, you'll see the final norm even closer to 1.
3. **Direction converges to the true feature.**
   The trajectory in the (c_0, c_1) plane hugs the circle and moves toward the point where essentially all mass is on the true feature ($c_0 \approx \|c(0)\|$, $c_1 \approx 0$).
4. **Loss → 0.**
   $J(c) \approx 10^{-6}$ is effectively zero, meaning the learned function matches the true function on the test distribution.

And as we worked out, at the limit:

- c*=(‖c(0)‖,0)c* = (‖c(0)‖, 0) (up to tiny numerical error),
- β^*=(1/‖c(0)‖,0)β^* = (1/‖c(0)‖, 0),
- So β^0*c0*=1β^0*c0* = 1, giving exactly fα(x)=ϕt(x)fα(x) = ϕt(x).

So you're good for 5(c): you've demonstrated empirically that GD on c drives you to the true-feature direction while keeping the radius essentially constant, and the loss collapses to ~0.

Whenever you're ready, upload the Jupyter notebook for Q5 and we'll walk through the remaining parts step by step.

You said:

Participation_B_rough_q_maml.ipynb File
Great. Let's go ahead and work through the remaining questions. Here is the notebook. I ran the cells up to the point where coding is required, and there are 2 TODO blocks. Tell me how to fill each in.

ChatGPT said:

Nice, let's finish off the two TODOs. Both of them are about **switching from MSE to logistic loss** in the classification setting.

From the markdown in the notebook, the logistic loss for labels
zi∈{−1,1}zi ∈ {−1, 1} and scores titi is

ℓi=log(1+e−ziti)ℓi = log(1 + e−ziti)

and the total loss is the sum (or mean) over i.
We'll implement that in PyTorch, using `my_sign` to turn the real-valued y into ±1 labels.

TODO 1: `meta_update_classification`

Around lines 36–58 of that code cell:

```
def meta_update_classification(w_t, feature_weights_t, n_train_meta, phi_type, d, k_idx, k_val, noise_prob):
    x_meta = generate_x(n_train_meta, 'uniform_random')
    features_meta = featurize(x_meta, phi_type=phi_type, d=d, normalize=True)
    features_meta_t = torch.tensor(features_meta)
    y_meta = generate_y(features_meta, k_idx, k_val)
    y_meta = add_label_noise(y_meta, noise_prob)
    # y_meta += np.random.normal(0, noise_prob, y_meta.shape)

    y_meta_t = torch.tensor(y_meta)
    y_meta_pred_t = features_meta_t @ w_t

    ###############################################################################
    # TODO: Complete the following code
    #
    # Hint: In regression, we have written:
    #   criterion = torch.nn.MSELoss(reduction = 'sum')
    #   loss = criterion( y_meta_t, y_meta_pred_t)
    # Now for classification, we need a different loss, defined in the text block.
    # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
    ###############################################################################
```

Fill the TODO with the **logistic loss**:

```
    # Convert targets to +/- 1 using my_sign
    z_meta_t = my_sign(y_meta_t)

    # Logistic loss: sum_i log(1 + exp(-z_i * t_i))
    loss = torch.sum(torch.log1p(torch.exp(-z_meta_t * y_meta_pred_t)))
    ###############################################################################
    ###############################################################################

    return loss
```

(Using `torch.log1p(torch.exp(...))` gives a numerically stable `log(1 + exp(x))`.)

TODO 2: inner-loop loss in `meta_learning_classification`

Later in the same cell, inside `meta_learning_classification`, you have:

```
with higher.innerloop_ctx(dm, opt_inner, copy_initial_weights=False, track_higher_grads=True) as (mod, opt):

    for j in range(num_gd_steps):
        y_pred = mod(features_t)

        ############################################################
        # TODO: Complete the following code
        #
        # Hint: In regression, we have written:
        #   loss = torch.mean((y_pred - y_t)**2)
        # Now for classification, we need a different loss, defined in the text block.
        # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
        ############################################################
        ############################################################

        opt_inner.zero_grad()
        loss.backward(retain_graph=True)
        opt.step(loss)
```

Again, we swap in the same logistic loss, but now using the inner-loop labels `y_t`:

```
        # Convert to +/- 1 labels
        z_t = my_sign(y_t)

        # Logistic loss: mean over inner-task datapoints
        loss = torch.mean(torch.log1p(torch.exp(-z_t * y_pred)))
        ############################################################
        ############################################################
```

You can use `torch.sum` instead of `torch.mean` if you want the exact same scaling as the meta-update function; either is fine as long as step sizes are reasonable.

Summary of what you changed

- **Meta-update classification loss** (`meta_update_classification`):
  `loss = Σ log(1 + exp(-z_i t_i))` with `z_i = my_sign(y_meta)` and `t_i = y_meta_pred`.
- **Inner-loop classification loss** (`meta_learning_classification` inner GD):
  `loss = mean log(1 + exp(-z_i t_i))` with `z_i = my_sign(y)` and `t_i = y_pred`.

Once you drop those two snippets in, the classification MAML section should run and produce the plots for Question 5(g,h).

You said:

Participation_B_q_maml.ipynb File
Great. Here were the results!

ChatGPT said:

Nice, you got everything running 😊

Since I can't see your actual plots from the notebook, I'll walk you through **what the expected behavior is** and how to translate that into answers for the written sub-questions 5(d)–5(h). You can then just check that your plots match this qualitative story and word things accordingly.

5(d) – Regression: meta-learned features vs all-ones vs oracle, and the spike at ntrain_post=32ntrain_post = 32

What the plot should look like

- **Oracle curve** (uses only the true features):
  Lowest loss overall; it's the ideal baseline.
- **Meta-learned feature weights**:
  Starts worse, but as meta-learning progresses, its test loss curve should **approach the oracle curve**, especially for small and moderate ntrain_postntrain_post.
- **All-ones feature weights**:
  Typically higher loss, especially when ntrain_postntrain_post is small; it uses all spurious features, so the inner learner overfits to noise/aliases.

Downward spike at ntrain_post=32ntrain_post = 32

- With many post-task datapoints, the inner learner has more information and can "average out" some of the spurious directions even when features aren't perfectly meta-tuned.
- In the code, 32 is often chosen near the **total number of informative features** or the dimensionality that matters, so at that point regression is particularly well-conditioned; the min-norm solution aligns more strongly with the true feature subspace, causing a sudden drop in test error.

So your written answer can say:

- Meta-learned weights outperform all-ones and approach the oracle.
- The spike at 32 arises because that dataset size makes the regression particularly well-posed, so the solution aligns better with true features.

5(e) – Why meta-learning improves performance (regression)

What happens to feature weights over meta-iterations

- Many feature weights are gradually driven **toward zero**.
- A small subset (those in the true support S) grow in magnitude or remain non-zero.
- The pattern roughly matches the oracle's "mask": important features stay; unimportant ones are suppressed.

Interpretation

- When spurious features have **small c_i**, the inner min-norm regression has to "spend" a lot of β-norm to use them, so it doesn't; it prefers the directions that meta-learning kept large.
- This effectively **reparameterizes** the model so that the inner learner is biased toward the true feature subspace, which is exactly why test loss approaches the oracle.

So for 5(e), explain:

> The meta-update gradually shrinks the weights on unhelpful features, which makes them expensive to use in min-norm regression. As a result, the inner learner focuses on true features, improving test performance and bringing the curve close to the oracle.

5(f) – Gradient-descent inner loop: num_gd_steps = 5 vs 1

With `num_gd_steps = 5`

- The inner learner gets multiple gradient steps, so it can make meaningful progress from the meta-initialized parameters toward a good task-specific solution.
- Meta-learning makes those initial parameters especially **adaptable**: after 5 steps, performance should be better than starting from a generic initialization.

With `num_gd_steps = 1`

- The inner learner gets only a single, small update.

- MAML's whole point is to make the initialization such that even **a few steps** work well; with 1 step you should still see some improvement from meta-learning, but:
    - The gain is smaller.
    - The curves may be noisier / closer to baseline, since one step can't fully exploit the good initialization.

So your answer:

- Yes, meta-learning improves performance at test time when `num_gd_steps = 5`: the meta-initialized network reaches lower loss than a non-meta-trained one.
- With `num_gd_steps = 1`, meta-learning can still help, but the effect is weaker; the initialization is optimized to make even one step useful, but one step is inherently limited in how far it can move.

5(g) – Classification: meta-learned vs all-ones vs oracle

This is the classification analogue of 5(d).

Expected behavior of the "classification error vs n_train_post" plot

- **Oracle** (logistic regression on only true features):
  Lowest classification error across ntrain_postntrain_post.
- **Meta-learned feature weights**:
  Error curve should be **below** the all-ones curve, especially when there are few post-task samples, and should **approach** the oracle performance as meta-learning progresses.
- **All feature weights = 1**:
  Higher error, especially at small ntrain_postntrain_post, because the inner logistic regression wastes capacity on noisy / alias features and is more prone to overfitting to spurious structure.

So you can write:

> Meta-learned feature weights significantly reduce classification error compared to the naïve all-ones initialization, particularly when only a small number of task-specific training points are available. Their performance tracks the oracle closely, although the oracle remains best because it uses perfect knowledge of the true feature support.

5(h) – Why some feature weights go to zero (classification)

The classification meta-learning run should show very similar qualitative behavior to the regression case:

- Over meta-iterations, **weights associated with irrelevant or alias features shrink toward zero**.
- Weights associated with truly predictive features stabilize at non-zero values.

Why that improves performance

- In logistic regression, features with very small weights contribute essentially no signal to the logits.
- By shrinking unhelpful feature weights, meta-learning:
    - NARROWS the effective feature space seen by the inner logistic learner,
    - REDUCES variance / overfitting to spurious correlations on small tasks,
    - ENCOURAGES the inner learner to update primarily in directions that historically reduced meta-loss.

So for 5(h), you can say:

> The meta-learning process prunes away uninformative feature directions by driving their weights toward zero. This makes the inner logistic regression effectively operate in a lower-dimensional subspace aligned with the true features, which explains the improved classification performance and the similarity to the oracle curve.

If you want, you can paste any specific plot or numeric summary and I can help you turn it into a polished, homework-ready paragraph for each subpart.